

PROYECTO INTERMODULAR

BASE DE DATOS – PROGRAMACIÓN

Persistencia en un juego de rol por turnos (Witcher RPG)

3^{er} Trimestre

Nombre del alumno/a: Aarón Hernández Moreno y Víctor de Lucas Fernández

Curso académico: 1º DAM G2

Tutor del proyecto: Carlos Galarón

ÍNDICE PAGINADO

1. JUSTIFICACIÓN DEL PROYECTO

2. INTRODUCCIÓN

3. OBJETIVOS

3.1. Objetivo general

3.2. Objetivos específicos

3.3. Objetivos funcionales

4. DESARROLLO

4.1. Fundamentación teórica

4.2. Materiales y métodos

4.3. Resultados y análisis

5. DIAGRAMA

6. CONCLUSIONES

7. LÍNEAS DE INVESTIGACIÓN FUTURAS

8. BIBLIOGRAFÍA

9. ANEXOS



1. JUSTIFICACIÓN DEL PROYECTO

Durante el segundo trimestre desarrollamos en la asignatura de Programación un juego de rol por turnos ambientado en el universo de The Witcher. El juego cuenta con un sistema de combate entre dos equipos de tres personajes cada uno, mecánicas de armas, hechizos y estados alterados, todo gestionado en memoria mediante objetos de Java.

El problema de ese diseño inicial es que toda la información desaparecía al cerrar el programa: cada vez que se ejecutaba el juego, los personajes volvían a su estado inicial y no quedaba ningún registro de las partidas jugadas. Para un juego real esto no es aceptable, ya que el jugador espera poder guardar su progreso y retomarlo más adelante.

Este proyecto intermodular surge precisamente de esa necesidad: dotar al juego de persistencia mediante una base de datos relacional en MySQL. La justificación es doble. Por un lado, académica, ya que integra los contenidos de las asignaturas de Bases de Datos y Programación en un único proyecto coherente. Por otro lado, práctica, porque reproduce un escenario habitual en el desarrollo de software profesional: separar la lógica de negocio de la capa de persistencia de datos.

La elección de MySQL como sistema gestor de base de datos se justifica por ser uno de los más utilizados en el ámbito profesional, por su gratuidad, por su amplia documentación y por su integración sencilla con Java a través del conector JDBC.

2. INTRODUCCIÓN

La presente memoria documenta el diseño e implementación de la base de datos del proyecto Witcher RPG, así como su integración con el código Java del juego. El objetivo es que el juego sea capaz de guardar el estado completo de una partida, cargarlo posteriormente, listar las partidas disponibles y borrar las que ya no se deseen conservar, todo ello mediante operaciones CRUD (Create, Read, Update, Delete).

El juego enfrenta a dos equipos: el bando bueno, formado por Geralt de Rivia, Yennefer de Vengerberg y Ciri; y el bando malo, La Cacería Salvaje, formado por Imlerith, Caranthir y Eredin. Cada personaje pertenece a una de tres clases (Guerrero, Mago o Sacerdote), cada una con mecánicas propias, y dispone de dos armas y uno o varios hechizos.

La base de datos debe reflejar fielmente esta estructura de objetos. Esto plantea un reto interesante de modelado: trasladar una jerarquía de herencia de Java (la clase abstracta Personajes con sus subclases) y unas relaciones de muchos a muchos (un personaje tiene varias armas, varios hechizos y participa en varios combates) a un modelo relacional normalizado.

A lo largo de la memoria se describe la fundamentación teórica del modelo, las herramientas y la metodología empleadas, y los resultados obtenidos, incluyendo el script SQL completo, las operaciones CRUD y la correspondencia entre las clases Java y las tablas de la base de datos.

3. OBJETIVOS

3.1. Objetivo general

Diseñar e implementar una base de datos relacional en MySQL que permita dotar de persistencia al juego Witcher RPG, integrándola con el código Java mediante JDBC para guardar, cargar, listar y borrar partidas.

3.2. Objetivos específicos

- Modelar mediante un diagrama entidad-relación las entidades del juego (personajes, armas, hechizos, estados y combates) y sus relaciones.
- Traducir la jerarquía de herencia de las clases de personajes (Guerrero, Mago, Sacerdote) a tablas relacionales mediante la técnica de tabla por subtipo.
- Resolver las relaciones de muchos a muchos mediante tablas intermedias (personaje-arma, personaje-hechizo, combate-personaje).
- Implementar las operaciones CRUD necesarias para gestionar partidas guardadas.
- Poblar la base de datos con los datos del catálogo del juego (personajes, armas y hechizos con sus estadísticas reales).
- Garantizar la integridad referencial de los datos mediante claves primarias, foráneas y restricciones.
- Documentar la correspondencia entre las clases Java y las tablas de la base de datos.

3.3. Objetivos funcionales

Los siguientes objetivos funcionales, recogidos en el enunciado del proyecto intermodular, se han cumplido totalmente. El juego es capaz de realizar todas las operaciones de persistencia exigidas a través de un menú durante su ejecución:

Objetivo funcional	Estado
Realizar operaciones CRUD a través de un menú durante la ejecución del programa.	✓ Cumplido totalmente
Comenzar una nueva partida: crear un personaje nuevo o retomar uno existente.	✓ Cumplido totalmente
Guardar automáticamente después de cada turno.	✓ Cumplido totalmente
Cargar una partida guardada anteriormente según el turno.	✓ Cumplido totalmente
Borrar partidas que ya no se quieran conservar.	✓ Cumplido totalmente
Listar las partidas disponibles para cargar.	✓ Cumplido totalmente

4. DESARROLLO

4.1. Fundamentación teórica

El modelo de datos se construyó a partir del análisis del código Java del juego. A continuación se describen las decisiones de diseño más relevantes.

Modelo entidad-relación

Se identificaron cinco entidades principales que se corresponden con las clases del juego: PERSONAJES, ARMAS, HECHIZOS, ESTADOS y COMBATE. Cada una se traduce en una tabla con sus atributos propios.

Tratamiento de la herencia

En el código, la clase abstracta Personajes tiene tres subclases: Guerrero, Mago y Sacerdote, cada una con atributos exclusivos (el Guerrero tiene defensa por adrenalina y probabilidad de bloqueo; el Mago tiene un multiplicador mágico; el Sacerdote tiene un bonus de sanación). Para representar esta herencia en el modelo relacional se utilizó la técnica de tabla por subclase: existe una tabla PERSONAJES con los atributos comunes y tres tablas adicionales (GUERRERO, MAGO, SACERDOTE) cuya clave primaria es a la vez clave foránea hacia PERSONAJES. De este modo, cada personaje tiene una fila en PERSONAJES y otra en la tabla de su clase.

Relaciones de muchos a muchos

El análisis del código reveló tres relaciones de muchos a muchos que no pueden representarse directamente en un modelo relacional y requieren tablas intermedias:

- **PERSONAJE_ARMA:** un personaje dispone de varias armas y cada arma puede pertenecer a varios personajes. El atributo equipada indica cuál arma lleva equipada actualmente (en Java es la diferencia entre la lista de armas disponibles y el arma equipada).
- **PERSONAJE_HECHIZO:** relaciona personajes con hechizos. El atributo cooldownActual guarda los turnos que le quedan a cada hechizo para volver a estar disponible, que en Java se almacenaba en un Map de cooldowns.
- **COMBATE_PERSONAJE:** resuelve la relación entre combates y personajes. Un combate guardado contiene seis personajes y un personaje puede aparecer en varios combates guardados.

Discriminadores en lugar de subtipos para armas y hechizos

Tanto las armas (cuerpo a cuerpo / a distancia) como los hechizos (daño directo, daño en el tiempo, curación directa, curación en el tiempo) tienen subtipos en el código Java. Sin embargo, esos subtipos no añaden atributos nuevos que haya que persistir: solo cambian el comportamiento de un método (polimorfismo). Por ello, en la base de datos no se crearon tablas de subtipo, sino que se utilizó un campo discriminador de tipo ENUM (el campo tipo en ARMAS y tipoHechizo en HECHIZOS) que indica de qué subtipo se trata. Al cargar desde la base de datos, el código usa ese campo para instanciar la subclase correcta.

Datos estáticos frente a datos dinámicos

Una decisión de diseño importante fue distinguir entre los datos que forman parte del catálogo del juego y los que se generan durante la partida. Las tablas PERSONAJES, ARMAS, HECHIZOS y sus relaciones contienen datos estáticos que se insertan al crear la base de datos. En cambio, las tablas COMBATE, COMBATE_PERSONAJE y ESTADOS contienen datos dinámicos que el juego va creando, actualizando y borrando durante su ejecución, por lo que arrancan vacías.

4.2. Materiales y métodos

Para el desarrollo del proyecto se utilizaron las siguientes herramientas y tecnologías:

- **MySQL:** sistema gestor de base de datos relacional donde se crean las tablas y se almacenan los datos.



- **phpMyAdmin:** interfaz web para administrar la base de datos, ejecutar consultas y exportar el script SQL.
- **Java (Eclipse IDE):** lenguaje y entorno donde está programado el juego.
- **Conector JDBC (mysql-connector-j):** librería que permite a Java conectarse a MySQL, gestionada mediante Maven en el archivo pom.xml.
- **Draw.io:** para diseñar el modelo entidad-relación antes de crear las tablas.

Metodología

La metodología seguida fue incremental. En primer lugar se analizó el código Java existente para identificar las entidades y relaciones. A continuación se elaboró el diagrama entidad-relación, que se fue refinando hasta corregir todas las inconsistencias detectadas (relaciones mal definidas, falta de cardinalidades, subtipos innecesarios). Después se tradujo el diagrama a sentencias CREATE TABLE en SQL y se poblaron las tablas estáticas con los datos del catálogo. Finalmente se definieron las operaciones CRUD que el juego ejecutará para gestionar las partidas.

Estructura de la base de datos

La base de datos consta de las siguientes tablas, agrupadas según su función:

Tabla	Tipo	Función
PERSONAJES	Principal	Datos comunes de los 6 personajes
GUERRERO / MAGO / SACERDOTE	Subtipo (ISA)	Atributos exclusivos de cada clase
ARMAS	Principal	Las 12 armas del juego
HECHIZOS	Principal	Los 10 hechizos del juego
ESTADOS	Principal (dinámica)	Estados activos (Quemadura, Veneno, Renovar)
COMBATE	Principal (dinámica)	Partidas guardadas
PERSONAJE_ARMA	Intermedia	Qué armas tiene cada personaje y cuál equipa
PERSONAJE_HECHIZO	Intermedia	Qué hechizos tiene cada personaje y su cooldown
COMBATE_PERSONAJE	Intermedia (dinámica)	Qué personajes participan en cada combate
JUGADORES	Principal (dinámica)	Datos del jugador: victorias, derrotas y contadores

Tabla	Tipo	Función
HISTORIAL	Principal (dinámica)	Acciones registradas de cada combate
LOGROS	Principal (estática)	Catálogo de los 13 logros disponibles
LOGROS_JUGADOR	Intermedia (dinámica)	Qué logros ha desbloqueado cada jugador

4.3. Resultados y análisis

El resultado del proyecto es una base de datos funcional con su script de creación, los datos del catálogo y las operaciones CRUD. A continuación se muestran los elementos más destacados.

Creación de las tablas principales

Las tablas se crean con sus claves primarias, tipos de datos adecuados y restricciones. Por ejemplo, la tabla PERSONAJES y la tabla de subtipo GUERRERO:

```
CREATE TABLE PERSONAJES (  
    ID_personaje INT NOT NULL AUTO_INCREMENT,  
    nombre VARCHAR(100) NOT NULL,  
    vidaMax INT NOT NULL,  
    vidaActual INT NOT NULL,  
    recursoMax INT NOT NULL,  
    recursoActual INT NOT NULL,  
    ataqueBase INT NOT NULL,  
    defensaBase INT NOT NULL,  
    poderMagico INT NOT NULL DEFAULT 0,  
    PRIMARY KEY (ID_personaje)  
);  
  
CREATE TABLE GUERRERO (  
    ID_personaje INT NOT NULL,  
    defensaAdrenalina INT NOT NULL,  
    probBloqueo DOUBLE NOT NULL,  
    PRIMARY KEY (ID_personaje),  
    FOREIGN KEY (ID_personaje) REFERENCES PERSONAJES(ID_personaje)  
);
```

Tablas intermedias

Las tablas intermedias usan una clave primaria compuesta por las dos claves foráneas, lo que garantiza que no se repita la misma combinación. Por ejemplo, PERSONAJE_ARMA:

```
CREATE TABLE PERSONAJE_ARMA (  
    ID_personaje INT NOT NULL,  
    ID_ARMA      INT NOT NULL,  
    equipada     BOOLEAN NOT NULL DEFAULT FALSE,  
    PRIMARY KEY (ID_personaje, ID_ARMA),  
    FOREIGN KEY (ID_personaje) REFERENCES PERSONAJES(ID_personaje),  
    FOREIGN KEY (ID_ARMA)      REFERENCES ARMAS(ID_ARMA)  
);
```

Datos del catálogo: personajes

Los seis personajes se insertan con las estadísticas reales del juego (ya ajustadas para que los combates no sean excesivamente largos):

Personaje	Clase	Vida	Recurso	Ataque/Def	P. Mágico
Geralt de Rivia	Guerrero	800	100	85 / 30	0
Yennefer	Mago	400	300	30 / 10	100
Ciri	Sacerdote	600	200	70 / 20	50
Imlerith	Guerrero	1000	50	90 / 35	0
Caranthir	Mago	500	280	25 / 10	90
Eredin	Sacerdote	750	200	60 / 25	60

Operaciones CRUD

Las cuatro operaciones CRUD se corresponden con momentos concretos del juego. La siguiente tabla resume cuándo se ejecuta cada una:

Operación	Cuándo ocurre en el juego	Tablas afectadas
CREATE	Al guardar una partida nueva o aplicar un estado	COMBATE, COMBATE_PERSONAJE, ESTADOS
READ	Al listar partidas o cargar una guardada	COMBATE, PERSONAJES, PERSONAJE_ARMA, PERSONAJE_HECHIZO, ESTADOS
UPDATE	Tras cada turno (vida, mana, cooldowns, arma)	PERSONAJES, PERSONAJE_HECHIZO, PERSONAJE_ARMA, ESTADOS, COMBATE
DELETE	Al borrar una partida o expirar un estado	ESTADOS, COMBATE_PERSONAJE, COMBATE

Un ejemplo representativo es la consulta READ que carga el estado completo de un personaje en una partida concreta, uniendo varias tablas mediante JOIN:

```
SELECT p.*, pa.ID_ARMA, a.nombre AS arma, pa.equipada,  
       ph.ID_HECHIZO, h.nombre AS hechizo, ph.cooldownActual  
FROM PERSONAJES p  
JOIN COMBATE_PERSONAJE cp ON cp.ID_personaje = p.ID_personaje  
LEFT JOIN PERSONAJE_ARMA pa ON pa.ID_personaje = p.ID_personaje  
LEFT JOIN ARMAS a ON a.ID_ARMA = pa.ID_ARMA  
LEFT JOIN PERSONAJE_HECHIZO ph ON ph.ID_personaje = p.ID_personaje  
LEFT JOIN HECHIZOS h ON h.ID_HECHIZO = ph.ID_HECHIZO  
WHERE cp.ID_COMBATE = 1 AND p.ID_personaje = 1;
```

Es importante el orden de borrado al eliminar una partida: primero se borra la tabla intermedia y después la principal, porque la integridad referencial impide borrar un combate del que todavía dependen filas en COMBATE_PERSONAJE.

```
DELETE FROM COMBATE_PERSONAJE WHERE ID_COMBATE = 1;  
DELETE FROM COMBATE WHERE ID_COMBATE = 1;
```

Correspondencia entre clases Java y tablas

La siguiente tabla muestra cómo se relaciona cada clase del juego con su tabla correspondiente en la base de datos, que es uno de los entregables exigidos:

Clase Java	Tabla BD	Observaciones
Personajes (abstracta)	PERSONAJES	Atributos comunes a todos
Guerrero	GUERRERO	PK = FK a PERSONAJES
Mago	MAGO	PK = FK a PERSONAJES
Sacerdote	SACERDOTE	PK = FK a PERSONAJES
Armas (+ subtipos)	ARMAS	Subtipo en campo ENUM tipo
Hechizos (+ subtipos)	HECHIZOS	Subtipo en campo ENUM tipoHechizo
Estados (Quemadura, Veneno, Renovar)	ESTADOS	Subtipo en campo ENUM tipoEstado
Combate	COMBATE	Una fila por partida guardada
Map de cooldowns	PERSONAJE_HECHIZO	El Map se traduce en filas
Lista de armas	PERSONAJE_ARMA	equipada distingue la equipada

Integración con Java mediante JDBC

La conexión con la base de datos se realiza mediante el conector JDBC, declarado como dependencia en el archivo pom.xml de Maven. El acceso a los datos se hace con sentencias preparadas (PreparedStatement) usando el carácter ? como marcador de posición para los parámetros, lo que evita errores de concatenación y protege frente a inyección SQL. Los momentos del juego en los que se interactúa con la base de datos son: al guardar una partida (INSERT de combate y sus personajes), tras cada turno (UPDATE de vida, recursos y cooldowns), al cargar una partida (SELECT con JOIN) y al borrar una partida (DELETE en el orden correcto).

5. DIAGRAMA

El modelo entidad-relación extendido (EER) del proyecto se elaboró en draw.io y pasó por dos versiones. En las páginas siguientes se muestran ambos diseños a tamaño completo para facilitar su lectura.

El primer diseño recoge el núcleo del juego: la entidad PERSONAJES con sus subtipos (Mago, Guerrero y Sacerdote), su relación con ARMAS (EQUIPAR) y HECHIZOS (USAR), los ESTADOS aplicables (Quemadura, Veneno y Renovar) y la entidad COMBATE.

El segundo diseño es la versión más completa y todavía en proceso de finalización. Amplía el modelo anterior incorporando la gestión del jugador y el progreso: las entidades JUGADOR, LOGROS (DESBLOQUEA) y ESTADISTICAS (GUARDA), junto con la tabla intermedia COMBATE_PERSONAJE que resuelve la participación de los personajes en cada combate.

5.2. Segundo diseño del modelo EER (en proceso)

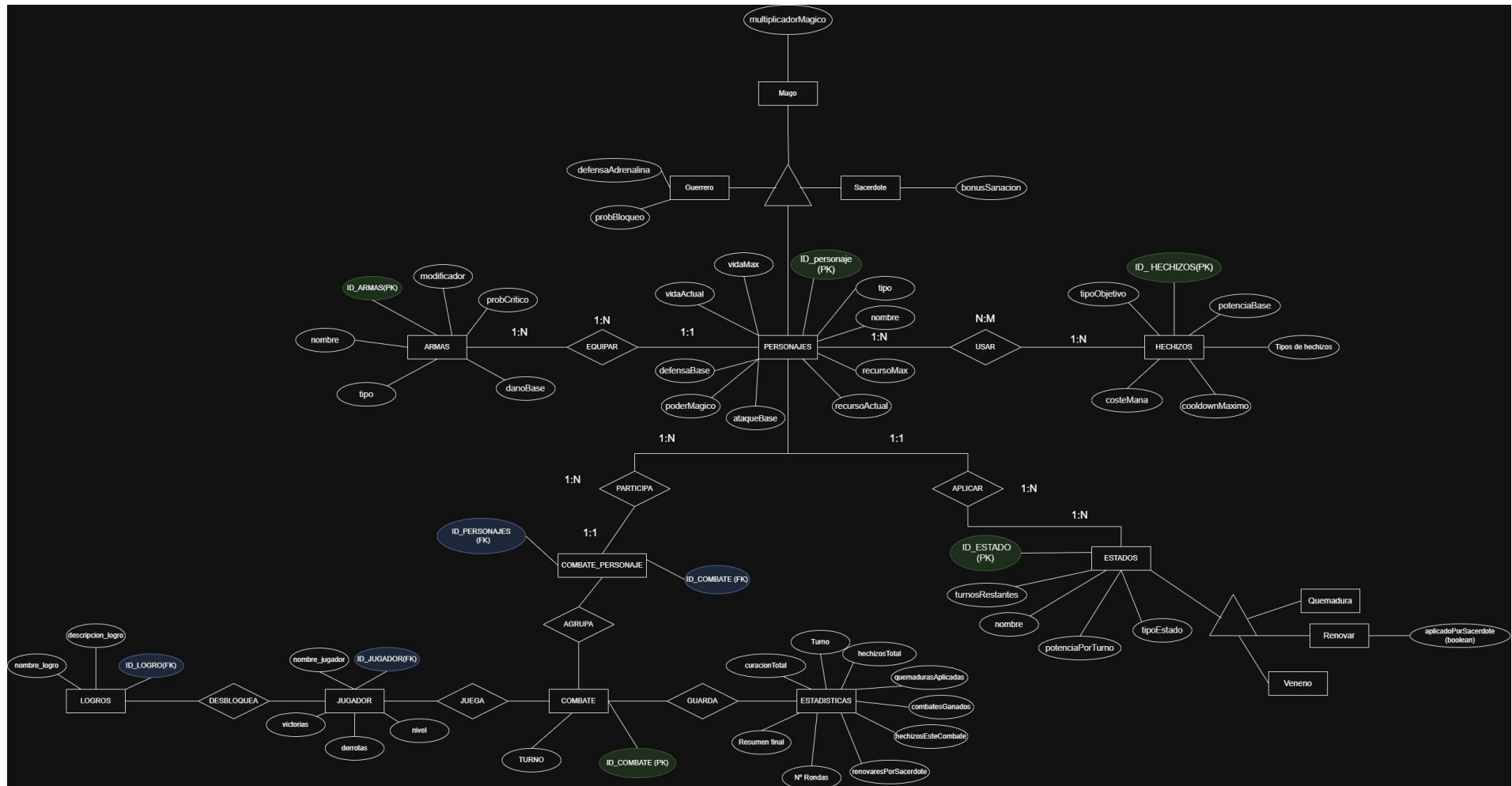


Figura 2. Segundo diseño del diagrama EER, versión ampliada con jugador, logros y estadísticas (pendiente de finalizar).

6. CONCLUSIONES

El proyecto ha cumplido el objetivo de dotar de persistencia al juego mediante una base de datos relacional en MySQL. Se ha conseguido un modelo normalizado que refleja fielmente la estructura de objetos del juego, resolviendo correctamente los retos de modelado que planteaba el código original.

Entre los aprendizajes más importantes destaca la traducción de conceptos de la programación orientada a objetos al modelo relacional: la herencia se resolvió con tablas de subtipo, las colecciones (listas y mapas) se convirtieron en tablas intermedias, y el polimorfismo de los subtipos que no añadían datos se sustituyó por campos discriminadores de tipo ENUM.

También se ha comprendido la importancia de la integridad referencial y del orden de las operaciones, especialmente en los borrados, donde respetar las dependencias entre tablas es imprescindible para no provocar errores.

Por último, la distinción entre datos estáticos (catálogo) y datos dinámicos (partidas) ha resultado clave para diseñar una base de datos coherente, en la que las tablas de partidas arrancan vacías y se van llenando a medida que el jugador juega.

7. LÍNEAS DE INVESTIGACIÓN FUTURAS

A partir de la base actual, el proyecto puede ampliarse en varias direcciones:

- Guardado automático tras cada turno, de modo que el progreso nunca se pierda ante un cierre inesperado del juego.
- Ranking global de jugadores ordenado por victorias o nivel, apoyado en la tabla JUGADORES ya creada.
- Historial detallado de partidas que registre todas las acciones relevantes de cada combate.
- Sistema de dificultad configurable almacenado en base de datos, que modifique la vida o el daño de los enemigos sin tocar el código.
- Persistencia completa del sistema de logros, vinculando los logros desbloqueados a cada jugador en la tabla LOGROS_JUGADOR.

8. BIBLIOGRAFÍA

Documentación oficial de MySQL. <https://dev.mysql.com/doc/>

Documentación oficial de phpMyAdmin. <https://www.phpmyadmin.net/docs/>

Documentación del conector MySQL para Java (JDBC).
<https://dev.mysql.com/doc/connector-j/en/>

Apuntes y materiales de las asignaturas de Bases de Datos y Programación, 1º DAM.

Documento de requisitos del proyecto intermodular proporcionado por el profesorado.

9. ANEXOS

Anexo I. Script SQL completo de creación de tablas e inserción de datos del catálogo (archivo proyecto_rpg.sql).

Anexo II. Diagrama entidad-relación del modelo de datos.

Anexo III. Sentencias CRUD comentadas para las operaciones de guardado, carga, listado y borrado de partidas.

Anexo IV. Código Java de la capa de acceso a datos (conexión JDBC y métodos de persistencia).